

What "Canvas" Means (and Why It Matters)

Canvas is simply a persistent, editable workspace beside this chat where we can design long-form artifacts together: architecture diagrams (in text), PRDs, system specs, build plans, and iteration notes. Instead of answers getting lost in chat history, this becomes your **living system blueprint**.

For Glazyr, canvas is the right place because you're not asking for a single answer — you're designing a platform.

What You Are Trying to Accomplish (Synthesis)

After analyzing both documents, here's the clear intent behind Glazyr:

Glazyr is not just:

- a browser extension
- a chatbot
- a web app

Glazyr is:

A **vision-first, interface-agnostic AI agent platform** that:

1. **Sees** digital interfaces (screenshots, DOM, UI trees)
2. **Understands** them using VLMs + semantic reasoning
3. **Acts** through pluggable executors (browser, APIs, OS, workflows)

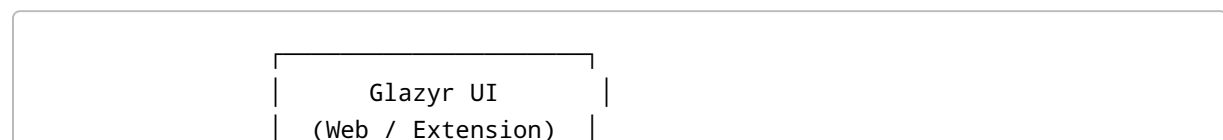
This mirrors how a human uses software — not APIs, but *interfaces*.

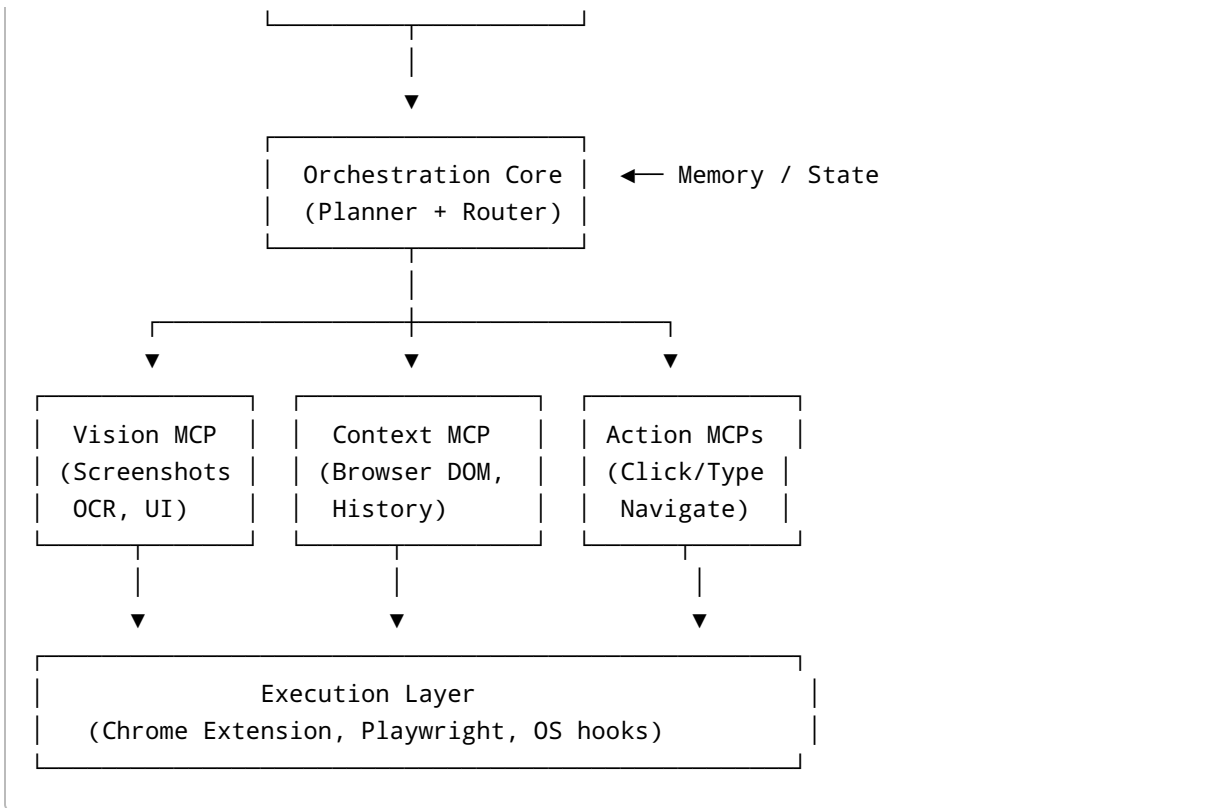
Your documents consistently emphasize: - Vision over DOM - MCP as the control plane - Decoupled Eyes / Brain / Hands - Local-first + privacy aware - Extension first, desktop later

This is a strong, coherent direction.

High-Level Architecture (Corrected & Hardened)

Below is a **robust, scalable architecture** that aligns with your PRD *and* avoids common agent pitfalls.





Components Explained (Plain English)

1. Website (glazyr.com)

Purpose: - Marketing - Auth & onboarding - Agent configuration - Developer docs

Tech: - Next.js / Astro - Auth (Clerk / Auth.js) - MCP registry UI

Important: The website is *not* the agent. It's the control plane.

2. Browser Extension (Primary Interface)

This is Glazyr's **eyes + hands** in the browser.

Responsibilities:

- Capture screenshots
- Read DOM (secondary signal)
- Track navigation + user intent
- Inject UI (side panel)

Architecture:

Manifest V3 Extension - Background service worker → state + messaging - Content scripts → DOM + events - Side panel UI → chat / commands

Key insight from your feasibility doc:

You **do not** need a Chromium fork for MVP.

Extensions + MCP already give you: - Full visual context - Automation - Cross-browser reach

3. MCP Servers (Yes, Multiple — and That's Good)

You are correct: **this should be multi-MCP.**

Minimum MCP Set (MVP)

A. Vision MCP

Handles: - Screenshot ingestion - OCR - UI element detection - Coordinate mapping

Tech: - VLM (GPT-4.1-V, Claude Vision, or local) - Tesseract fallback

B. Browser Action MCP

Handles: - Click(x,y) - Type(text) - Scroll - Navigate

Backends: - Chrome Extension - Playwright (headless fallback)

C. Context / Memory MCP

Handles: - Page history - Interaction timeline - Short-term task memory

This prevents the "goldfish agent" problem.

Post-MVP MCPs

- OS-level MCP (desktop)
 - Workflow MCP
 - API MCP
 - Personal Knowledge MCP
-

4. Orchestration Core (The Brain)

This is the **most important piece**.

Responsibilities: - Task decomposition - Tool selection - Retry logic - Ambiguity handling - Safety rails

Design rules: - Stateless execution - Explicit tool contracts - No UI logic

Think of this as **Kubernetes for agents**.

Data Flow (Cleaned & Corrected)

```
graph TD
    A[User Input (Voice/Text)] --> B[Orchestrator]
    B --> C[Screenshot + Context Snapshot]
    C --> D[Vision MCP → UI Map]
    D --> E[Planner → Action Plan]
    E --> F[Action MCP]
    F --> G[Loop until goal achieved]
```

Key correction vs naive agents: - Vision is *authoritative* - DOM is supportive, not primary - Actions are validated visually

Why This Architecture Is Strong

✓ Vision-first (future proof) ✓ MCP-native (composable) ✓ Extension-deployable (low friction) ✓
Desktop-expandable ✓ Investor-credible ✓ Technically defensible

This is **not** a wrapper. This is a platform.

Recommended Build Order (90-Day Plan)

Phase 1 (Weeks 1–4)

- Chrome extension (screenshots + side panel)
- Vision MCP (basic OCR + element detection)
- Action MCP (click/type via extension)

Phase 2 (Weeks 5–8)

- Orchestrator MVP
- Memory MCP
- Multi-step tasks

Phase 3 (Weeks 9–12)

- Polished UI
 - Website + onboarding
 - Developer docs
-

Next Steps I Can Do With You

- Draw a **formal MCP contract spec**
 - Design the **extension manifest + message bus**
 - Create a **system diagram for investors**
 - Define **MVP vs moat features**
 - Turn this into a **technical whitepaper**
-

Orchestrator Design: Planner, Retries, and Safety Rails

This section expands the **Orchestration Core**, because this is where most agent systems fail.

The orchestrator is **not** a chatbot and **not** a tool wrapper. It is a deterministic control system that supervises probabilistic models.

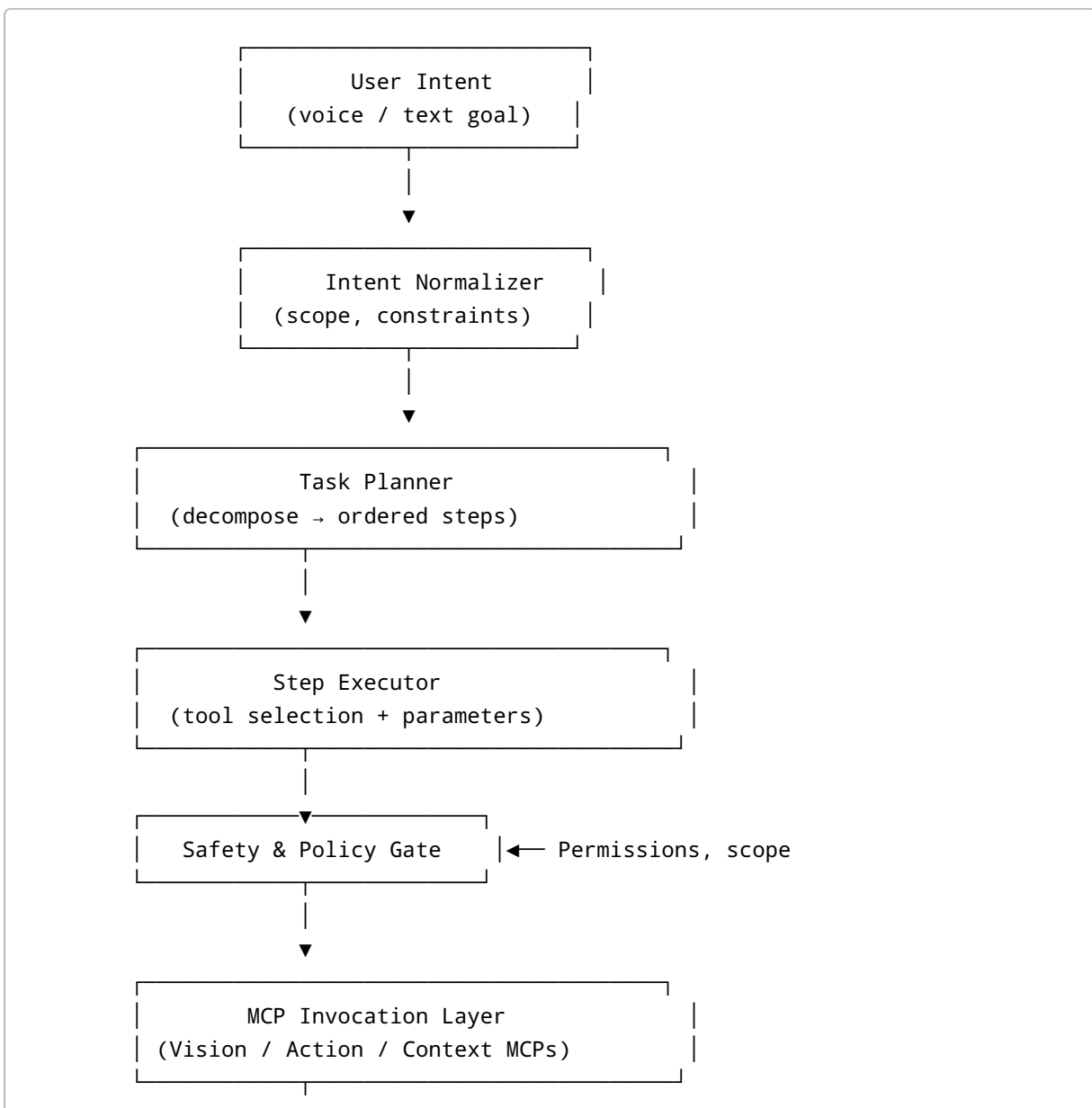
Mental Model (Important)

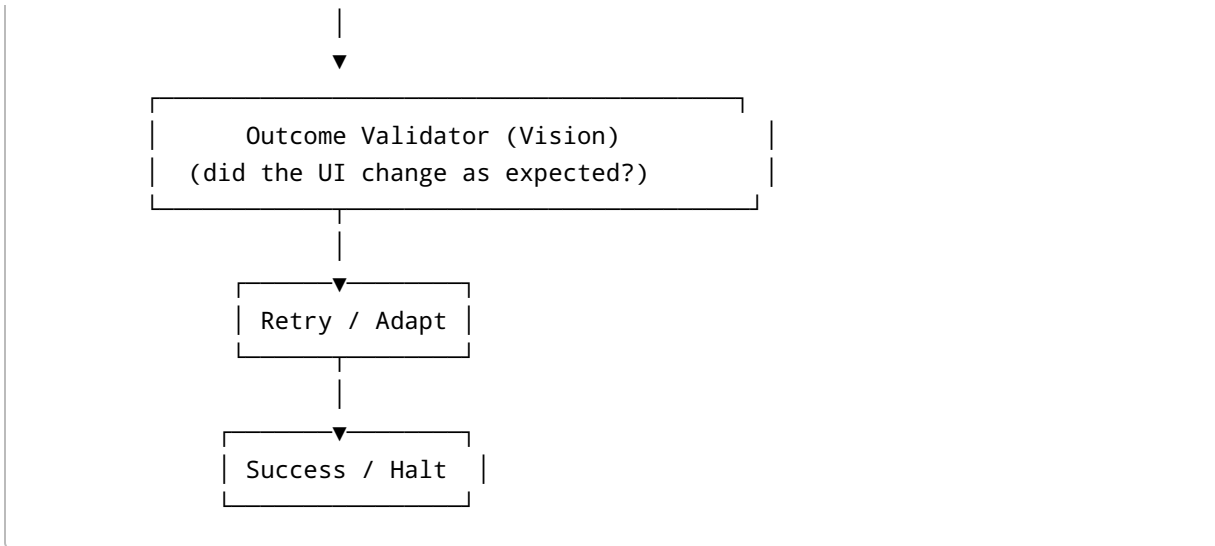
Think of the orchestrator as **air-traffic control**, not the pilot.

- LLMs propose actions
- MCPs execute actions
- The orchestrator decides *whether* actions are allowed, repeated, or stopped

If the orchestrator reasons *instead of* governing, you get: - infinite loops - hallucinated success - silent failure

Orchestrator High-Level Diagram





Core Components Explained

1. Intent Normalizer

Purpose: - Reduce vague human input into **bounded objectives**

Example:

"Fill out this form"

Becomes: - Target: current page - Allowed actions: type, click - Stop condition: form submitted OR error shown

This is your **first safety rail**.

2. Task Planner (LLM-assisted, not autonomous)

Responsibilities: - Break goal into explicit steps - Each step must be *verifiable*

Good plan: 1. Locate email field 2. Type email 3. Locate submit button 4. Click submit

Bad plan: - "Complete the signup"

Rule:

If a step cannot be visually verified, it is invalid.

3. Step Executor

Responsibilities: - Select the correct MCP - Provide strict parameters

Example:

```
action: click
coordinates: (x,y)
confidence: 0.92
```

No natural language allowed past this point.

4. Safety & Policy Gate (Critical)

This is where you prevent disasters.

Checks: - Is this action within user-granted permissions? - Is this site / domain allowed? - Is this a destructive action?

Rules: - Destructive actions require confirmation - Cross-domain actions require re-authorization

This gate runs **before every action**.

5. Outcome Validator (Vision-First)

After action execution: - Take a fresh screenshot - Compare against expected state

Examples: - Button disappeared - Modal appeared - Error message visible

This replaces brittle DOM assertions.

6. Retry & Adapt Loop

Retries are **structured**, not blind.

Retry types: - Parameter adjustment (different coordinates) - Alternate element (second-best match) - Re-plan (step failed twice)

Hard limits: - Max retries per step (e.g. 3) - Max total steps

If exceeded → HALT + explain.

Failure Modes You Are Actively Avoiding

Failure	Why it happens	How Glazyr avoids it
Infinite loops	No stop condition	Explicit halt rules
False success	No validation	Vision-based checks
Tool chaos	LLM chooses tools freely	Tool routing enforced
Silent errors	No reporting	Failure surfaced to user

Design Rules (Non-Negotiable)

- 1. **The orchestrator never clicks**
- 2. **The orchestrator never sees raw pixels**
- 3. **The orchestrator never trusts the LLM**
- 4. **Every action must be reversible or confirmed**

If you follow these, Glazyr will feel calm, intelligent, and safe.

Why This Is a Moat

Most agents: - Let the model drive - React after failure

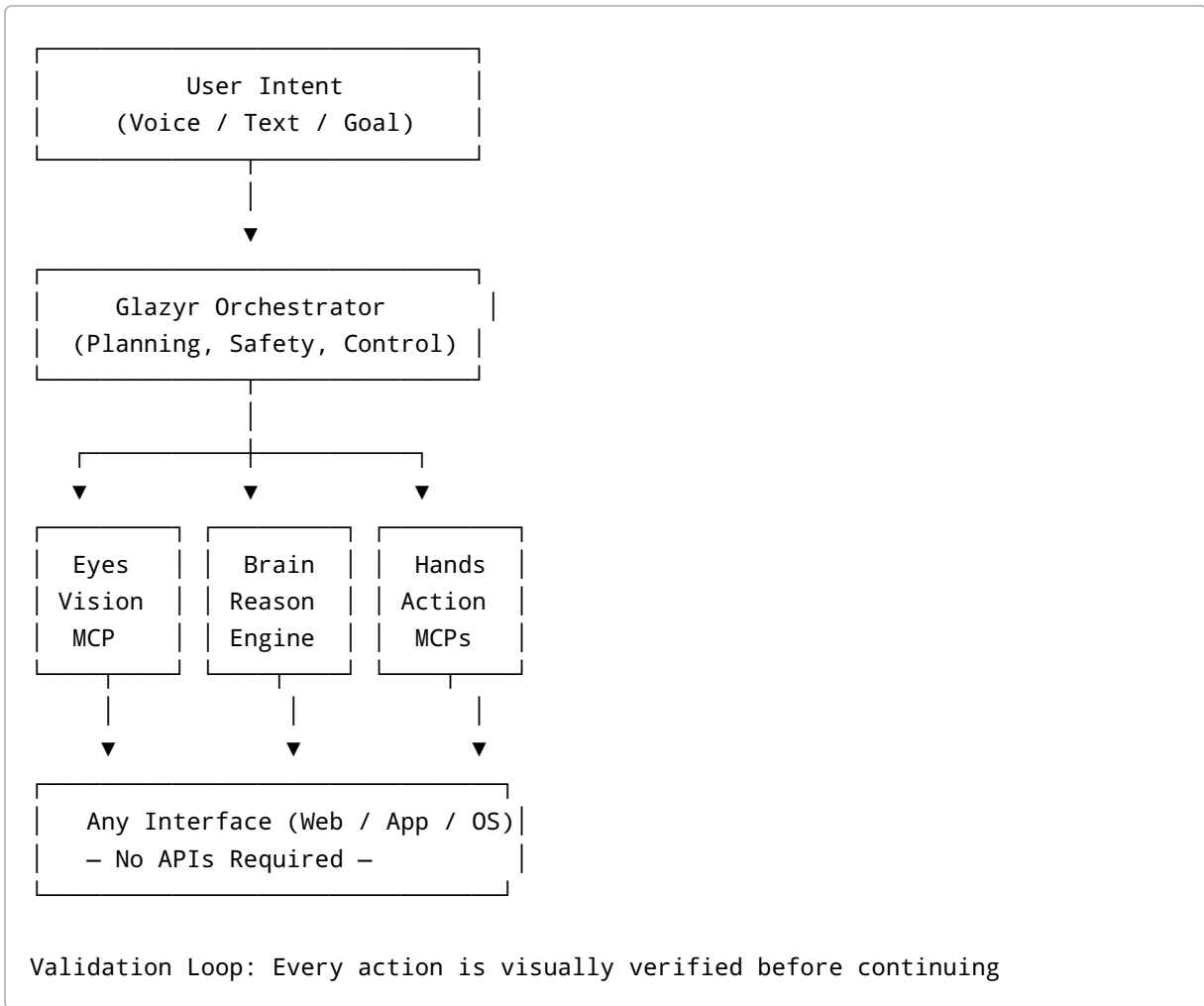
Glazyr: - Governs first - Executes second - Verifies always

That difference is architectural — and hard to copy.

Investor-Safe System Diagram & 60-Second Narrative

This section is designed to explain **why Glazyr is different** to a non-technical but sophisticated audience (investors, advisors, partners) in under one minute.

One-Page Investor Diagram



The 60-Second Narrative (Scriptable)

"Most AI agents today only work where APIs exist. They're blind to real interfaces and brittle outside controlled environments.

Glazyr is different.

Glazyr is a *vision-first AI agent* that works the way a human does — by seeing the screen, understanding what's on it, and acting safely.

We separate the system into **Eyes, Brain, and Hands**.

The Eyes see any interface through screenshots and vision models. The Brain plans and governs actions. The Hands execute clicks, typing, and navigation through modular control servers.

What makes Glazyr defensible is the orchestrator. It doesn't just generate actions — it verifies every step visually, enforces safety rules, and retries intelligently when something changes.

This means Glazyr works across websites, apps, and environments **without APIs**, and scales as interfaces evolve.

In short: most agents talk to software. **Glazyr uses it."**

Why Investors Immediately Understand This

This framing communicates:

- **Market expansion** → works where APIs don't
- **Technical moat** → orchestration + vision loop
- **Platform potential** → MCP-based extensibility
- **Enterprise readiness** → safety, verification, control

No deep ML explanation required.

Competitive Contrast (One Slide)

Category	Typical AI Agents	Glazyr
Interface	APIs / DOM	Vision-first
Reliability	Optimistic	Verified
Safety	Prompt-based	Architectural
Extensibility	Monolith	MCP platform
Long-term moat	Model access	System design

Why This Scales Over Time

- Models improve → Glazyr improves automatically
- Interfaces change → Glazyr adapts visually
- New domains → add MCPs, not rewrites

This positions Glazyr as **infrastructure**, not a feature.

If you want next, we can: - Turn this into a **single-slide visual diagram** - Write a **pitch-deck version** (problem → solution → moat) - Map this narrative to **valuation logic** - Stress-test this against known competitors

Just tell me which direction you want to go.